

Windows Python 개발 환경 가이드

ChatGPT 대화 정리 · 2026년 8월 기준
Anaconda vs Python · MLOps/ML Engineer 환경 구성

요약

2026년 기준 Windows에서 Python을 시작할 때 가장 권장되는 구성은 **Python 공식 설치 + uv/venv + VS Code + Git + Docker** 입니다. Anaconda Full은 입문용으로 편리하지만 현업 MLOps 흐름과는 다소 거리가 있습니다. CUDA GPU 딥러닝을 본격적으로 할 계획이면 Miniconda를 추가하는 방식이 현실적입니다.

최종 추천 순위

순위	선택	적합한 경우
🥇 1순위	Python 공식 설치 + uv + venv	MLOps/개발 중심, 실무와 동일한 구조 학습
🥈 2순위	Miniconda	GPU/CUDA, 데이터 사이언스, ML 실험
🥉 3순위	Anaconda Full	입문·Jupyter 중심 빠른 시작 (현업 흐름과 조금 다름)

Part 1. Windows Python 세팅 (단계별)

2026년 기준 권장 구성: Python 공식 설치 · VS Code · venv · pip 또는 uv

1. Python 설치

- 공식 홈페이지(python.org) → Windows Installer (64-bit) 다운로드
- 설치 시 반드시 체크: **Add python.exe to PATH**
- 설치 확인: `python --version` 또는 `py --version`

2. VS Code 설치

- Extension: Python (Microsoft), Pylance, Black Formatter, Jupyter

3~6. 프로젝트 & 가상환경

```
mkdir D:\Project\Python
cd D:\Project\Python
python -m venv .venv

# PowerShell 활성화
.venv\Scripts\Activate.ps1

# CMD 활성화
.venv\Scripts\activate.bat
```

7~9. 패키지 관리

```
python -m pip install --upgrade pip
pip install pandas numpy
pip freeze > requirements.txt
pip install -r requirements.txt
```

10. VS Code 인터프리터

Ctrl+Shift+P → **Python: Select Interpreter** → .venv 선택

11~12. 실행 & Git

```
python main.py

# .gitignore
.venv/
__pycache__/
.vscode/
*.pyc
```

13. 추천 개발 패키지

black (포맷), ruff (린터), pytest (테스트), requests, pandas, numpy, matplotlib, jupyter

14. uv 사용 (최신 권장)

```
uv venv
uv pip install pandas
```

가상환경 생성.패키지 설치가 pip보다 빠르고 재현성이 좋습니다.

추천 구성 (Windows + VS Code)

항목	추천
Python	공식 Python 최신 안정 버전
IDE	VS Code
가상환경	venv
패키지 관리	uv (권장) 또는 pip
포매터 / 린터 / 테스트	Black / Ruff / Pytest
버전 관리	Git + GitHub

Part 2. Python 버전 & Anaconda 안정성

Anaconda Python 3.13.9는 Stable한가?

네. Anaconda에 포함된 Python 3.13.9는 충분히 안정적인(Stable) 버전입니다. 3.13.9는 3.13 계열의 9번째 패치(Maintenance) 릴리스로, 새 기능 추가보다 버그 수정·보안 패치 중심입니다.

Stable 최신 버전 비교 (2026년 8월 기준)

Python 3.14.x가 최신 안정 메이저 버전이지만, '최신'과 '가장 추천'은 항상 같지 않습니다.

항목	Python 3.13.9	Python 3.14.x
안정성	★★★★★ (매우 검증됨)	★★★★☆ (상대적으로 새로움)
라이브러리 호환성	★★★★★	★★★★☆☆ (일부 미지원 가능)
ML/Data Science	★★★★★ (권장)	★★★★☆☆ (주의 필요)
신규 프로젝트	★★★★★	★★★★☆

Anaconda 내 Stable Python 버전

Anaconda에서 가장 안정적으로 많이 쓰이는 버전: **Python 3.12.x**, **Python 3.11.x**. 3.13.9도 stable하지만 ML/GPU 라이브러리 호환성을 고려하면 3.12.x가 더 무난한 경우가 많습니다.

현업 Python 버전 사용 현황

Python	사용 현황
3.11	★★★★★ 매우 많음
3.12	★★★★★ 증가 중
3.13	★★★★☆ 신규 프로젝트
3.10 이하	기존 프로젝트 유지

최신보다 한두 단계 이전 버전 선호 — PyTorch, TensorFlow, CUDA, GPU Driver, ML 라이브러리 호환성 때문.

Part 3. Anaconda vs Python 공식 설치

항목	Anaconda Full	Miniconda	Python + venv/uv
설치 크기	대 (~3GB+)	소	최소
시작 속도	빠름 (올인원)	보통	보통
패키지 관리	conda	conda	pip / uv
실무 유사도	낮음~보통	보통~높음	높음
GPU/CUDA	강함	강함	별도 설정
MLOps/배포	조금 다름	가능	가장 가까움

개인 학습용으로 Anaconda?

운영 반영이 아닌 **개인 학습·분석** 목적이라면 Anaconda는 편리합니다 (Jupyter, pandas 등 즉시 사용). Python 생태계(venv, pip, requirements.txt)를 깊이 이해하려면 Python 공식 설치가 더 유리합니다.

셋 중 하나만 설치한다면

1순위 Python 공식 설치 + venv — ★★★★★

2순위 Miniconda — ★★★★★☆ (GPU/ML 실험 시)

3순위 Anaconda Full — ★★★☆☆ (빠른 입문용)

여러 OS(Windows/Mac) 경험 관점

```
# Windows
python -m venv .venv
.venv\Scripts\activate
pip install pandas sqlalchemy

# Mac (거의 동일)
python3 -m venv .venv
source .venv/bin/activate
pip install pandas sqlalchemy
```

실무 서버(Linux, Airflow, ETL)에서는 python3 + venv + pip + requirements.txt 구조가 일반적입니다. Anaconda(conda)는 별도 생태계이며, 플랫폼 이동성은 좋지만 실무와는 거리가 있습니다.

추천 학습 순서

1. **1단계 (Windows):** Python 3.13.9 + VS Code + venv + pip

2. **2단계:** 같은 프로젝트를 Mac에서 Git clone 후 재구축
3. **3단계:** Miniconda, Docker, Poetry, uv 등으로 확장

Part 4. MLOps / ML Engineer 환경 (2026년 기준)

현업 표준: Python + 가상환경(uv/venv/conda) + Docker + Git + VS Code

1. ML Engineer 환경 구조

```
개발 PC (Windows/Mac/Linux)
├─ Python 3.11 ~ 3.13
├─ Virtual Environment (uv / venv / conda)
├─ VS Code / PyCharm
├─ Git
└─ Docker → Training / Serving Environment
```

→ 로컬에서 실험하고 Docker로 재현

2. 가상환경 사용 현황

1순위: uv

```
uv init ml-project
cd ml-project
uv add torch pandas scikit-learn
```

pip보다 빠름 · 재현성 좋음 · Python 버전 관리 가능

2순위: conda / mamba (GPU, CUDA, 데이터 사이언스)

```
conda create -n ml python=3.12
conda activate ml
```

3순위: venv + pip (Backend ML API, FastAPI, MLOps 서비스)

3. Anaconda 현업 사용 형태



Anaconda Full Distribution 보다



Miniconda / Micromamba 선호

4. ML 프로젝트 구조 예시

```
customer-ai-model/  
├─ data/  
├─ notebooks/experiment.ipynb  
├─ src/train.py, predict.py  
├─ models/  
├─ Dockerfile  
├─ pyproject.toml  
├─ uv.lock  
└─ README.md
```

환경 : Python 3.12 + uv + PyTorch + MLflow + Docker

5. MLOps 배포 구조

Developer PC → GitHub → CI/CD → Docker Image
→ Kubernetes → Model Serving

영역	기술
버전관리	Git
패키지	uv / poetry
컨테이너	Docker
실험관리	MLflow
Pipeline	Airflow / Kubeflow
Serving	FastAPI / Triton
Container 운영	Kubernetes

6. Data Engineer → MLOps Engineer 경로

Hadoop, Airflow, ETL, Linux, DB, Infra 운영 경험이 있다면 **Data Engineer → ML Engineer → MLOps Engineer** 방향이 자연스럽습니다.

7. 추천 Windows 개인 PC 구성

Windows 11

- └─ Python 3.13.9 (또는 3.12)
- └─ uv
- └─ VS Code
- └─ Docker Desktop
- └─ Git
- └─ Jupyter Extension

ML 실험: uv venv + PyTorch, Pandas, Scikit-learn, MLflow

최종 결론

-  Anaconda Full — 입문용으로는 좋지만 현업 흐름과 조금 다름
-  Miniconda — ML/GPU 환경에서는 여전히 좋은 선택
-  Python 공식 설치 + uv + Docker — 현재 MLOps 흐름과 가장 가까움

하나만 설치한다면: **Python 공식 설치 + uv 환경을 추천.**

CUDA 기반 딥러닝(PyTorch GPU)을 본격적으로 할 계획이면 그때 Miniconda를 추가하는 방식이 가장 현실적입니다.

출처: ChatGPT 공유 대화 · <https://chatgpt.com/share/6a71349d-dc00-83ee-b7c0-642f7556f8a1>

정리일: 2026년 8월 4일